

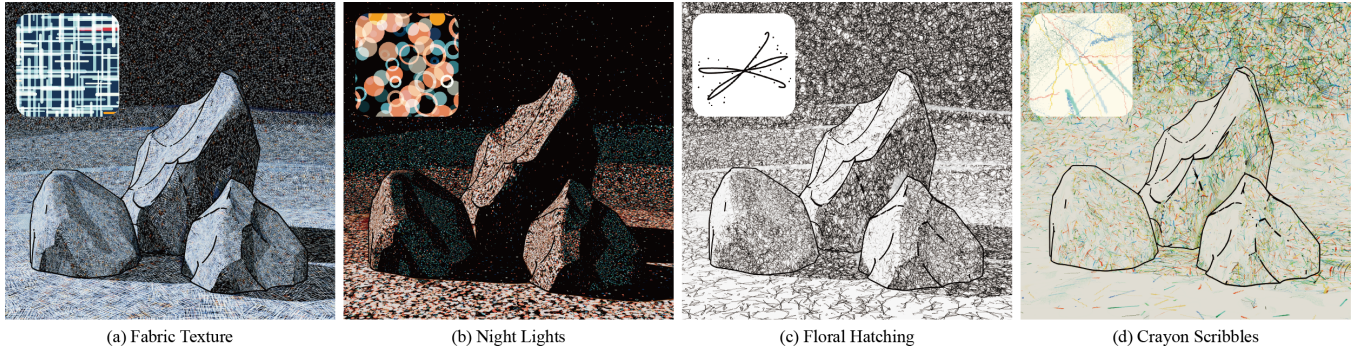
# Generative Tonal Art: An Artistic Rendering Workflow using Programmable TAMs

Yun-Chen Lee

National Yang Ming Chiao Tung University  
Hsinchu, Taiwan  
ycllee@arch.nycu.edu.tw

June-Hao Hou

Graduate Institute of Architecture, National Yang Ming  
Chiao Tung University  
Hsinchu, Taiwan  
jhou@arch.nycu.edu.tw



**Figure 1: Generative tonal art rendering results. (a) Using 2D primitives (lines) generated in p5.js. (b) Using circles with p5.js blend mode (SCREEN). (c) Using hand-drawn image assets. (d) Using custom strokes from the p5.brush library.**

## Abstract

In this work, we explore a workflow that bridges the gap between generative art and non-photorealistic rendering (NPR) for the creative coding community. Our goal is to enable artists to design customized rendering styles for their 3D scenes using p5.js, a user-friendly and open-source programming language, without the need for advanced shader programming. We introduce the concept of Generative Tonal Art Maps (GTAMs) as a mediating layer linking 2D texture design to 3D NPR rendering. The workflow involves two parts: (1) In p5.js, GTAMs are generative value scales composed of multiple parametric layers. Each layer allows users to design the appearance and behavior of sampled points. (2) The generated GTAM image can then be imported into Blender through our tonal art rendering pipeline, allowing for NPR rendering with dynamic light sources. We have provided a proof of concept based on various textures generated by p5.js (see Figure 1) and envision an open-source platform where artists can share their own GTAMs and generative parameters.

## CCS Concepts

• Computing methodologies → Non-photorealistic rendering.

## ACM Reference Format:

Yun-Chen Lee and June-Hao Hou. 2025. Generative Tonal Art: An Artistic Rendering Workflow using Programmable TAMs. In *SIGGRAPH Asia 2025 Posters (SA Posters '25)*, December 15–18, 2025, Hong Kong, Hong Kong. ACM, New York, NY, USA, 3 pages. <https://doi.org/10.1145/3757374.3771537>

## 1 Introduction and Motivation

In non-photorealistic rendering (NPR), a toon shader is a type of shader designed for 3D scenes that simplifies lighting effects by using textures with fewer colors and tones. Compared with 3D painting tools, which require users to manually draw within the 3D environment [Tada 2025; Yao et al. 2024], toon shaders provide a simple yet effective way to create 2D illustration-like styles [Barla et al. 2006; DoubleGum 2018; Dédouze 2021; Praun et al. 2001; Wyatt 2024]. However, existing approaches only allow users to adjust color parameters and do not provide an easy way to create new textures. We found that p5.js is a powerful tool for exploring 2D textures and developing unique aesthetics [Kislev 2021; Wang 2023]. Its ease of use and rapid visual iteration make it a promising medium that enables artists to create customized textures. Inspired by the concept of generative art, we propose a new approach to texture generation by extending Tonal Art Maps (TAMs) [Praun et al. 2001] into Generative Tonal Art Maps (GTAMs).

## 2 Approach

Our workflow consists of two parts: (1) In p5.js, users define procedural drawing behaviors for each layer, and then our algorithm automatically generates GTAMs as image outputs (see Figure 4). (2) The generated GTAMs are then imported into Blender through our tonal art rendering pipeline to achieve real-time NPR rendering.

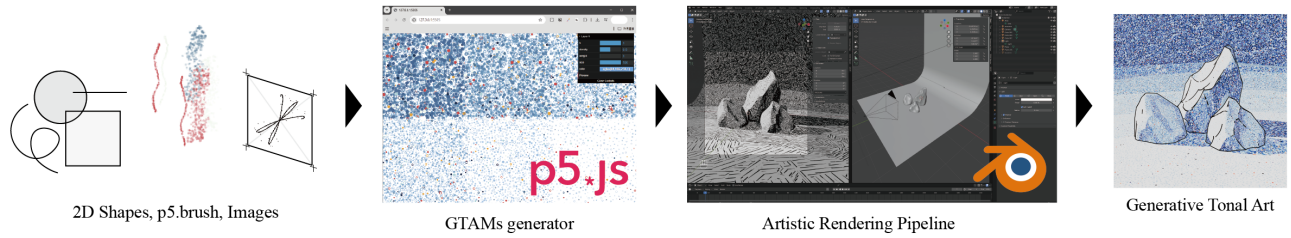


Figure 2: Stylization workflow.

## 2.1 Programmable GTAMs using P5.js

According to Praun et al. [2001], we highlight three key properties for composing GTAMs.

- (1) **Uniformity:** The strokes must be uniformly distributed across different tone levels. This property better reflects the characteristics of hand-drawn art and avoids visual repetition when tiling.
- (2) **Stroke Nesting Property:** The darker tone must be generated by adding new strokes onto lighter tones.
- (3) **Seamlessness:** The maps must be seamless to avoid visible boundaries when textures are tiled.

First, to achieve uniform point distribution, we implement sample elimination [Yuksel 2015] instead of traditional additive sampling. Our algorithm first generates a large set of random points, and then sample elimination is applied to gradually remove them to reduce density, producing a Poisson-like uniform pattern. During the drawing phase, these points are replaced with input primitives (e.g., 2D shapes, brush strokes generated by *p5.brush* [Campos 2024], or images) to generate GTAMs. Unlike the additive approach, which requires a predefined radius and has an unclear stopping condition, making it difficult for users to precisely control the visual outcome, this method sorts points by their elimination order, allowing users to access a specific subset of samples through simple array slicing without regenerating points from scratch. As a result, under a fixed total stroke count, this method can easily control the visual effect by adjusting the minimum and maximum density values (see Figure 3).

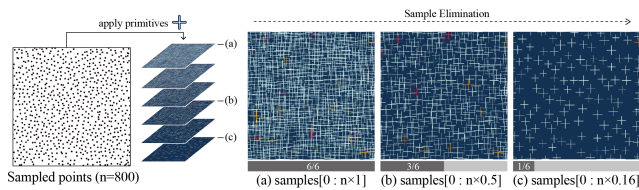


Figure 3: Density control by array slicing

Additionally, we extend the GTAMs generator with tone controls, including multi-color gradients, stroke size variation, and random color or rotation jitter (see Figure 4). To achieve the stroke nesting property, each stroke is assigned a unique seed, ensuring it maintains the same appearance across tone levels.

For seamlessness, we define a boundary offset threshold based on stroke size. Within this offset region near the texture borders, strokes are duplicated onto the opposite side to ensure continuous

tiling. The probability of generating strokes within this duplicated region is reduced to prevent over-dense areas along the edges.

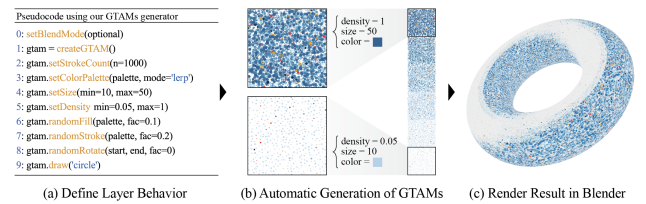


Figure 4: GTAMs generation process.

## 2.2 Tonal Art Rendering in Blender

We implement the rendering pipeline in Blender (see Figure 5). The GTAM is stored as a texture and integrated into a toon-shader framework, where tonal layers are sampled according to diffuse lighting. To avoid repetitive patterns when tiling, we use camouflage-like patches based on noise and Voronoi patterns. Each patch is assigned a local rotation, producing natural variations in stroke direction and reducing visible tiling artifacts.

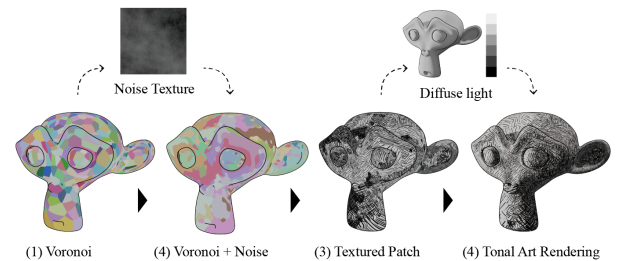


Figure 5: Rendering pipeline in Blender.

## References

- Pascal Barla, Joëlle Thollot, and Lee Markosian. 2006. X-toon: An extended toon shader. In *Proceedings of the 4th international symposium on Non-photorealistic animation and rendering*. 127–132.
- Alejandro Campos. 2024. p5.brush.js. <https://p5-brush.cargo.site/>. Accessed: 2025-10-16.
- DoubleGum. 2018. Komikaze: Toon Shaders & Assets Pack (Stylized Npr). <https://superhivemarket.com/products/komikaze>. Accessed: 2025-10-16.
- Dédouze. 2021. How this 2D/3D animation was made - Introduction to Blender greasepencil and tips for beginners. [https://www.youtube.com/watch?v=salFT8\\_j0LQ](https://www.youtube.com/watch?v=salFT8_j0LQ). Accessed: 2025-10-16.

- Orr Kislev. 2021. Obscure Cameras. <https://www.fxhash.xyz/project/obscure-cameras>. Accessed: 2025-10-16.
- Emil Praun, Hugues Hoppe, Matthew Webb, and Adam Finkelstein. 2001. Real-time hatching. In *Proceedings of the 28th annual conference on Computer graphics and interactive techniques*. 581.
- Gaku Tada. 2025. Deep Paint. <https://gakutada.com/dp-en/>. Accessed: 2025-10-16.
- Aluan Wang. 2023. After The Cave. <https://aluanwang.com/short-form/after-the-cave/>. Accessed: 2025-10-16.
- Alan Wyatt. 2024. Tradigitalist Toolbag. <https://tradigital.gumroad.com/>. Accessed: 2025-10-16.
- Zhihao Yao, Qirui Sun, Beituo Liu, Yao Lu, Guanhong Liu, Xing-Dong Yang, and Haipeng Mi. 2024. InkBrush: A Sketching Tool for 3D Ink Painting. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*. 1–15.
- Cem Yuksel. 2015. Sample elimination for generating poisson disk sample sets. In *Computer Graphics Forum*, Vol. 34. Wiley Online Library, 25–32.